

SHIELD Test Plan

Sep 28, 2026 · Bryson Noble

System for High Integrity Elimination of Low-Altitude Drones (SHIELD) — Software Test Plan, Milestone 1

1. Project Title, Team Members

System for High Integrity Elimination of Low-Altitude Drones (SHIELD)

- Bryson Noble (SWE) - bnoble2023@my.fit.edu
- Sophia Versage (AEE) - sversage2024@my.fit.edu
- Edward Ledwith (AEE) - eledwith2023@my.fit.edu
- Nick Jacobson (AEE) - njacobson2023@my.fit.edu
- Aryaa Vijay (AEE) - avijay2024@my.fit.edu
- Miguel Mohammed-Mueses (AEE) - mmohammedmue2023@my.fit.edu
- Isabelle Chamberlain (AEE) - ichamberlain2023@my.fit.edu
- Joshua Huser (AEE) - jhuser2023@my.fit.edu
- Will Emerson (AEE) - wemerson2023@my.fit.edu
- Tharushi Pelendagama (AEE) - tpelendagama2023@my.fit.edu

AEE Faculty Advisor: Madhur Tiwari, PhD - mtiwari@fit.edu

Client: Firat Irmak, PhD - Aerospace Engineering Department, Florida Institute of Technology

2. Introduction

Purpose. This plan defines how each software requirement in the SHIELD Requirements Document (AVS.01 through AVS.05.01) will be verified, and by which automated test script. Every requirement traces to at least one test case, and every test case records its result as PASS, FAIL, or ABORT.

Scope. The plan covers only the avionics software: onboard target detection, tracking, confidence monitoring, real-time processing, and the drone-to-Ground Control System (GCS) command link. Airframe, propulsion, structural, and guidance (AER, STR, GNC) requirements are part of the AEE team's test plan and are out of scope here.

Objectives.

1. Verify every functional, performance, and interface requirement through automated, repeatable test scripts.
2. Run the same scripts first in emulation (Unity virtual camera feeding the Python pipeline on a PC), then on the onboard hardware (Jetson Orin Nano + e-CAM25_CUNOX) in later milestones.
3. Keep a log for every run so each verdict can be audited after the fact.

3. Test Items

Six requirements are under test, plus their parent system requirement SYS.03 (autonomous onboard detection), which the detection tests exercise directly.

Requirement	Summary	Type	Verification Method
AVS.01	Detect a target within 20 m	Functional	Analysis, Test
AVS.02	Recognize and track a target with a minimum span of 0.25 m	Functional	Analysis, Test
AVS.03	Maintain at least 25% confidence en route to the target	Performance	Demonstration, Test
AVS.04	Process detection within 33.33 ms of receiving a frame	Performance	Analysis, Test
AVS.05	Receive Engage and Abort commands from the GCS	Interface	Demonstration, Test
AVS.05.01	Send a heartbeat to the GCS at 1 Hz	Interface	Analysis, Test

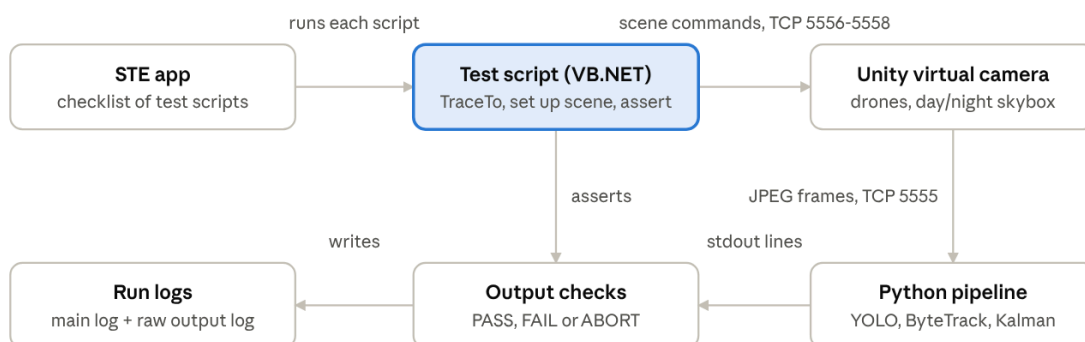
The software items exercised are the Python pipeline in the SHIELD repository: `video_source.py` (frame input), `object_detection.py` (YOLO detection, ByteTrack IDs, per-track Kalman filter), `gcs_ui.py` (operator interface, Engage/Abort, confidence floor), and `config.py` (thresholds and timeouts).

4. Test Approach

All software tests run through STE (SHIELD Test Environment), an in-house harness that turns each requirement into a scripted, repeatable test case.

Each test case drives the Unity scene, then asserts on the pipeline output

Unity and Python are relaunched fresh for every test case



STE test flow · 6 parts

The test script sets up the Unity scene, the Python pipeline reports what it sees on stdout, and the script's checks judge that output against the requirement.

Test scripts. Each script lives under Test Scripts/<SUBSYSTEM>/ and follows one shape: BeginTest(), one RunTestCase(AddressOf TCxx) per test case, then EndTest(). Each test case calls TraceTo("<requirement>") once per requirement it verifies, so coverage is recorded in the log rather than kept by hand.

Independent test cases. Before each test case, the harness closes and relaunches the Unity player and the Python pipeline, waits up to 30 s for the video link, and then waits a configurable startup delay (default: 15 s). Every test case starts from the same empty scene and never depends on a previous one.

Scene control. Test cases drive the scene over TCP: EditVirtualEnvironment switches day/night; InstDrone spawns a Quad, Toad, or BumbleBee drone at the given coordinates; and DespawnAllDrones clears them.

Result model.

- **PASS:** the test case ran to completion and every check held.
- **FAIL:** the test case ran to completion, but at least one check showed the requirement was not met. A failed check does not stop later checks.
- **ABORT:** an unhandled error (setup timeout, missing component, bug) stopped the test case, so nothing was verified either way.

The overall script result is the worst-case result: any ABORT outranks a FAIL, which outranks a PASS.

Logs. Each run writes <TestName>.log (header, traced requirements, every check, summary) and <TestName>_python_output.log (timestamped raw pipeline output) to <LogDirectory>/<TestName>/<yyyy-MM-dd_HH-mm-ss>/.

5. Test Cases

At least two test cases cover every required feature in the Requirements Document: one with usual inputs and one with unusual inputs designed to break it. Three test cases are scripted and runnable in emulation today; the rest are planned for Milestones 2 and 3 as the required features come online.

Required feature	Usual-input tests	Unusual-input tests
Target detection	T-01, T-03	T-04, T-09, T-10
Target tracking	T-05	T-10, T-12
Confidence monitoring	T-01, T-06	T-09, T-12
Real-time processing	T-02	T-02 at 3 targets, T-11
GCS command and control	T-07, T-08	T-11, T-13

Test	Script/test case	Requirements	Status
T-01 Range sweep detection	AVS/AVS_Detection_Test.vb TC01	AVS.01, AVS.02, AVS.03, SYS.03	Scripted

Test	Script/test case	Requirements	Status
T-02 Detection latency under load	AVS/AVS_Detection_Test.vb TC02	AVS.04	Scripted
T-03 Autonomous detection	SYS/SYS_Requirements_Test.vb TC01	SYS.03	Scripted
T-04 Environment and airframe variety	New, AVS/AVS_Detection_Test.vb TC03	AVS.01, AVS.02	Planned, Milestone 2
T-05 Multi-target track identity	New, AVS/AVS_Tracking_Test.vb TC01	AVS.02	Planned, Milestone 2
T-06 Confidence on approach	New, AVS/AVS_Tracking_Test.vb TC02	AVS.03	Planned, Milestone 2
T-07 Engage and Abort commands	AVS/AVS_GCS_Link_Test.vb TC01	AVS.05	Placeholder, Milestone 3
T-08 GCS heartbeat rate	AVS/AVS_GCS_Link_Test.vb TC02	AVS.05.01	Placeholder, Milestone 3
T-09 Non-drone decoy	New, AVS/AVS_Edge_Case_Test.vb TC02	AVS.03, SYS.03	Planned, Milestone 2
T-10 Frame edge and close range	New, AVS/AVS_Edge_Case_Test.vb TC03	AVS.01, AVS.02	Planned, Milestone 2
T-11 Video link loss	New, AVS/AVS_Edge_Case_Test.vb TC04	AVS.04, AVS.05.01	Planned, Milestone 3
T-12 Target lost mid-track	New, AVS/AVS_Tracking_Test.vb TC03	AVS.02, AVS.03	Planned, Milestone 2
T-13 Invalid operator commands	AVS/AVS_GCS_Link_Test.vb TC03	AVS.05	Planned, Milestone 3

T-01 Range sweep detection

- **Requirements:** AVS.01, AVS.02, AVS.03, SYS.03
- **Setup:** fresh Unity scene, day skybox, no drones; Quad drone prefab.
- **Procedure:**
 - With the scene empty, watch for 3 s and confirm no drone is reported.
 - Spawn one Quad at 1, 5, 10, 15, and 20 m along the camera's line of sight, one at a time.

- c. For each placement, check for a detection within 10 s, then record the lowest confidence over the next 3 s.
- d. Despawn, wait 2s, and move to the next range.
- e. Repeat the sweep at two off-axis offsets, (0.25, 0.15) and (-0.2, -0.3), and re-check the empty scene after each sweep.
- **Expected outcome:** all 15 placements are detected as drone, every confidence reading is at least 0.25, and all 4 empty-scene checks report nothing.
- **Alternative outcomes:** a missed detection at long range (FAIL, AVS.01); confidence dipping below 0.25 (FAIL, AVS.03); a detection in the empty scene (FAIL, false positive); the video link not confirmed within 30 s (ABORT).
- **Analysis for AVS.02:** the Quad prefab's rotor-tip span must be measured in Unity and shown to be 0.25 m or less; thus, a detection at 20 m establishes the minimum target size.

T-02 Detection latency under load

- **Requirements:** AVS.04
- **Setup:** fresh scene, pipeline launched with `--profile`, which prints average per-stage timing every 30 frames.
- **Procedure:**
 - a. Spawn Quads at 10 m one at a time, up to 3 in the scene, waiting 2 s after each.
 - b. After each spawn, read the most recent `detect=` time from the profile output.
 - c. Clear the scene and repeat the batch once more (6 readings in total).
- **Expected outcome:** every reading is below 33.33 ms, with 1, 2, and 3 targets in view.
- **Alternative outcomes:** detect time at or above 33.33 ms (FAIL); no profile line within 30 s (FAIL).
- **Note:** emulation measures the PC's GPU. The same script must be rerun on the Jetson Orin Nano in hardware mode before AVS.04 is closed.

T-03 Autonomous detection

- **Requirements:** SYS.03
- **Procedure:** spawn one Quad at (0, 5, 15), about 15.8 m away, with no operator input, and check for a drone detection within 20 s.
- **Expected outcome:** the pipeline reports the target on its own; any detection within 20s is a PASS.

T-04 Environment and airframe variety (planned)

- **Requirements:** AVS.01, AVS.02
- **Procedure:** repeat the T-01 sweep with the night skybox (`EditVirtualEnvironment(Night)`), then with the Toad and BumbleBee prefabs in day.
- **Expected outcome:** same pass criteria as T-01 for every lighting and airframe combination.
- **Why:** T-01 uses only the Quad in daylight, so it cannot show the detector generalizes beyond one target shape and one lighting condition.

T-05 Multi-target track identity (planned)

- **Requirements:** AVS.02 (track)
- **Procedure:** spawn 3 drones at distinct positions and record every `target id=` line for 10 s.

- **Expected outcome:** exactly 3 track IDs appear, and each stays with the same target for the whole window, with no ID swaps.
- **Needs:** a new check, e.g., `AssertDistinctTrackIds(className, count, durationSeconds)`, in `Common_Test_Checks.vb`.

T-06 Confidence on approach (planned)

- **Requirements:** AVS.03
- **Procedure:** spawn a drone at 20 m and move it toward the camera to 1 m, recording confidence the whole way.
- **Expected outcome:** confidence remains at or above 0.25 throughout the approach.
- **Needs:** drone motion scripting in Unity (a Milestone 2 task); today, a spawned drone cannot move, so T-01 checks confidence only at fixed ranges.

T-07 Engage and Abort commands (placeholder)

- **Requirements:** AVS.05
- **Procedure:** select a detected track, send Engage, then send Abort, and repeat Engage with the target below 25% confidence.
- **Expected outcome:** the drone logs `ENGAGE` issued for track T-xx, then `ABORT`; the low-confidence Engage is refused.
- **Today:** Engage and Abort are button clicks in `gcs_ui.py`, with no command channel to the drone, so this case can only be demonstrated manually until a scriptable GCS link is available.

T-08 GCS heartbeat rate (placeholder)

- **Requirements:** AVS.05.01
- **Procedure:** record the timestamps of at least 10 consecutive heartbeat messages.
- **Expected outcome:** every interval is 1.0 s within a tolerance still to be set (proposed ± 0.1 s).
- **Today:** each frame request acts as an implicit liveness check (2s timeout); there is no distinct 1 Hz heartbeat message to measure yet.

T-09 Non-drone decoy (planned)

- **Requirements:** AVS.03, SYS.03
- **Procedure:** spawn a balloon (needs a balloon prefab in Unity) at 5, 10, and 20 m with no drone present.
- **Expected outcome:** each balloon is reported as a `balloon`, never as a `drone`, and is never offered as an engageable target.
- **Why:** a misclassified decoy is the false positive AVS.03's confidence monitoring exists to catch.

T-10 Frame edge and close range (planned)

- **Requirements:** AVS.01, AVS.02
- **Procedure:** spawn a Quad half outside each frame edge, then at 0.5 m so it fills most of the frame.
- **Expected outcome:** each partially visible or oversized drone is still detected, and its track retains a single ID.

- **Why:** a target entering the field of view, or one about to be intercepted, is rarely centered and whole.

T-11 Video link loss (planned)

- **Requirements:** AVS.04, AVS.05.01
- **Procedure:** with a drone detected, stop the Unity stream for 5s, then restore it.
- **Expected outcome:** the pipeline detects the link loss within its 2s read timeout, the operator sees the loss, and detection resumes within 30 s of the stream returning, without a restart.
- **Why:** a stale frame treated as current is the failure of AVS.04's real-time requirement is meant to prevent.

T-12 Target lost mid-track (planned)

- **Requirements:** AVS.02, AVS.03
- **Procedure:** track a drone, hide it behind an occluder for 10 frames, then for 40 frames.
- **Expected outcome:** after 10 frames, the track keeps its original ID; after 40 frames (over the 30-frame TRACK_MAX_AGE), the track is dropped, and the operator sees a degraded-track warning.
- **Needs:** a Unity scene command that hides a drone without destroying it.

T-13 Invalid operator commands (planned)

- **Requirements:** AVS.05
- **Procedure:** send Engage with no track selected, Engage twice in a row, Abort with nothing engaged, and Abort, then Engage on the same track.
- **Expected outcome:** the first three are refused or ignored with a logged reason and no state change; the last engages normally.
- **Why:** an operator under time pressure will press the wrong button, and the drone must never act on a malformed command.

6. Requirements Traceability

Four of seven requirements have scripted coverage in emulation today; AVS.05 and AVS.05.01 wait on the GCS command link. Every requirement still needs a hardware run on the Jetson Orin Nano in Milestone 3.

Requirement	Parent	Verification Method	Test cases	Emulation coverage	Remaining work
AVS.01	SYS.03	Analysis, Test	T-01, T-04, T-10	Scripted	Night and airframe variety (T-04)
AVS.02	SYS.03	Analysis, Test	T-01, T-05, T-10, T-12	Partial	Measure Quad prefab span; track identity (T-05)

Requirement	Parent	Verification Method	Test cases	Emulation coverage	Remaining work
AVS.03	SYS.03	Demonstration , Test	T-01, T-06, T-09, T-12	Partial	Moving-target approach (T-06); test flight
AVS.04	SYS.03	Analysis, Test	T-02, T-11	Scripted	Rerun on Orin Nano hardware
AVS.05	SYS.01	Demonstration , Test	T-07, T-13	Not scripted	Scriptable GCS command channel; test flight
AVS.05.01	AVS.05	Analysis, Test	T-08, T-11	Not scripted	Distinct 1 Hz heartbeat message
SYS.03	—	Test	T-01, T-03, T-09	Scripted	Rerun on Orin Nano hardware

7. Test Environment

Testing runs in two stages: emulation on a Windows PC now, then the same scripts against the onboard hardware from Milestone 3.

Item	Emulation (Milestones 1-2)	Hardware (Milestone 3 onward)
Compute	Windows 11 PC with NVIDIA GPU (CUDA)	NVIDIA Jetson Orin Nano
Camera	Unity "SHIELD Virtual Camera" player, JPEG over TCP port 5555	e-CAM25_CUNOX (AR0234 global shutter)
Pipeline launch	<code>python __main__.py 0 --source unity -u --profile</code>	Platform mode 1 (Hardware), not yet implemented
Detection model	Custom drone/balloon YOLO; yolo11n stand-in until the yolo26s retrain lands	Final model chosen from on-device benchmarks
Test harness	STE app (WinUI 3) + STE_Test_Solution (VB.NET), both from the repo	STE on the PC, driving the Orin over the network

Item	Emulation (Milestones 1-2)	Hardware (Milestone 3 onward)
Unity	Editor 6000.3.11f1; standalone player built to Test Tools/SHIELD Virtual Camera/	Unity still supplies frames for hardware-in-the-loop runs before flight
Software	Python 3.11+, Ultralytics, OpenCV, PyTorch	Same pipeline, GPU-accelerated on the Orin

All configuration lives in config.py (confidence floor 0.25, TRACK_MAX_AGE 30 frames, 2s link timeout). Each run's log records the machine name and platform, so emulation and hardware results are never mixed.

8. Risks and Contingencies

The largest risk is that emulation passes overstate what the hardware will do; every requirement is therefore rerun on the Orin before it is closed.

Risk	Effect on testing	Contingency
Emulation runs on a desktop GPU, not the Orin	T-02 latency passes may not hold on the drone	Rerun T-02 in hardware mode; fall back to a smaller model (e.g., YOLO26n) if it fails
T-02 checks a 30-frame average, but AVS.04 bounds each frame	A few slow frames can hide inside a passing average	Print per-frame maximum and 99th-percentile detect time; assert on those too
The model is partly trained on Unity renders	Detection in the Unity scene may score higher than on real footage	Add a --source file run on recorded real drone video before hardware tests
Placeholder test cases (T-07, T-08) report PASS with no checks	The run summary overstates coverage	Report them as NOT VERIFIED, or exclude them from the overall result, until real checks exist
No drone motion, occlusion, or balloon prefab in Unity yet	T-06, T-09, T-12 cannot run	Add these scene commands in Milestone 2; until then, verify by demonstration
Unity must close through its own QUIT command (forced kills have crashed the GPU driver)	A hung player ABORTs that test case	Already handled: the case is marked ABORT, and the run continues; close the window before the next run
Requirement wording drift between the doc and the scripts	Traces may point to an outdated requirement	Align before Milestone 2: scripts use AVS-01, where the doc uses AVS.01, and the AVS-02 script comment still says 0.25 m ²

Risk	Effect on testing	Contingency
		cross-sectional area instead of 0.25 m span
GCS link and heartbeat not implemented	AVS.05 and AVS.05.01 untestable	Define the command and heartbeat message format in the Design Document; T-07, T-08, T-11, T-13 follow

9. Success Criteria

A requirement is verified when every test case that traces to it passes in emulation and on the Orin hardware, with logs to prove it.

1. Every requirement AVS.01 through AVS.05.01 traces to at least one test case with a real check, not a placeholder.
2. Every scripted test case passes in emulation on three consecutive runs, with no ABORTs.
3. T-01 through T-06 and T-09 through T-12 pass again with the pipeline running on the Jetson Orin Nano.
4. Requirements marked Demonstration (AVS.03, AVS.05) are also shown in a test flight, with the GCS log saved.
5. Each verified requirement links to the run folder (<TestName>/<timestamp>/), which contains its passing logs.

References

- [SHIELD repository](#): pipeline, STE harness, and test scripts
- [SHIELD Requirements Document](#) (AVS.01 through AVS.05.01)